

BURNER TRUST NETWORK

Product and Engineering Handoff

Prototype demo complete - canonical implementation brief

Implementation instruction

Preserve the settled user experience and invariants. Choose the simplest proven technical implementation. Escalate only choices that materially alter product behavior, privacy, safety, scope, or irreversible cost.

1. Executive Direction

Build a private, opt-in community network where real-world relationships form a navigable trust graph and short typed bulletins help people offer, request, organize, collaborate, appreciate, and ask for introductions.

The graph is the primary experience. The bulletin board is the useful, structured layer on top of the graph. The product should help trusted communities coordinate without turning into another noisy social network, marketplace, messaging app, or public reputation system.

Implementation should proceed without requiring the product owner to decide every low-level technical detail. When several implementations satisfy this document, choose the simplest proven approach.

2. Product Principles

1. The graph is home.
2. Relationships require consent.
3. Trust is private, directional, and viewer-owned.
4. Visibility is controlled by the viewer and author, not global popularity.
5. Bulletins are short, typed, useful, and temporary.
6. Existing messaging systems handle conversation and follow-up.
7. No public reputation score, rating, follower count, or engagement game.
8. The system works offline and recovers cleanly.
9. Safety controls are quiet, private, and effective.
10. Complexity must earn its place.

3. Scope

Included

- Supabase magic-link authentication.
- Minimal identity: immutable internal ID, unique handle, display name, optional avatar and contact fields.
- Mutual connections and private directional trust.
- Interactive graph with viewer-controlled visibility.
- Typed bulletins: Offer, Request, Event, Collaboration, Appreciation, Network Update, and directed Introduction Request.
- Saved Board Views and one Notify Me query.
- Web Push notifications.
- Offline cached state and queued mutations.
- Private reports, blocking, dismissal, hiding authors, and operator moderation.
- First-class API with the same authorization rules as the PWA.
- Deactivation and GDPR erasure.

Explicitly excluded from v1

- Native messaging or comments.
- Likes, reactions, ratings, reviews, or public reputation.
- Payments or transactions.
- Public people search.
- Groups.
- Native mobile applications.
- User-facing analytics.
- End-to-end encryption.
- Automatic community-wide punishment based on reports.

4. Core User Experience

Entry and identity

A user joins through an opaque, revocable invitation link or an approved entry flow. Authentication uses a magic link. Email and display name are required. A unique handle provides stable identification and sharing, but it is not the primary display identity.

The profile is intentionally small. There are no traditional profile pages. Person nodes and compact person cards provide only information the viewer is authorized to see.

Graph home

After authentication, the user lands on their graph. It should feel like a living map of their community rather than a directory.

The graph represents the full visible network the viewer is authorized to traverse. The product model should not impose an arbitrary depth cap. Performance may use caching, projections, incremental loading, clustering, or layout optimization, provided behavior remains conceptually complete and understandable.

People visible only as topology may appear as Private nodes. Hidden information must never be sent to the client merely to be concealed by the UI. Graph filters use only fields and relationships the viewer is permitted to use. There is no people search.

Connections and trust

A connection means: "I know this person and would introduce them into my community." Connections require explicit acceptance.

The connection is shared. Trust strength is private and directional. One person's trust value is never exposed to the other person or third parties. Unset trust is distinct from zero trust. Trust controls the viewer's experience; it is not a score assigned to a person.

Bulletins

Bulletins are short structured posts designed to make useful community coordination discoverable without becoming a social feed.

- Type
- Headline
- Short message
- Optional tags
- Optional location
- Author
- Visibility
- Created, updated, expiry, and archive timestamps

Bulletins have no drafts in v1. They are created active, may be edited, and later expire or are archived. Editing does not reset expiry. URLs may be detected in message text; there is no separate required link field.

Appreciation is permitted positive noise, but it must not become a targeted rating or reputation mechanism.

Introduction Requests are directed only to a direct connection who permits them. There is no in-product forwarding chain. Multi-hop introductions require explicit new connections and a new request at each step.

Board and Views

The Board is secondary to the graph. It shows bulletins the viewer is authorized to access and has not filtered out. Saved Views are persistent. Temporary search and filters may be layered on the active View. Search applies only to visible bulletin content, tags, and locations; never people.

Suggested defaults include Everything, Offers and Requests, Events, My Bulletins, and Dismissed. Default sorting is recent. Dismissal is viewer-local and does not alter the bulletin for anyone else.

Notify Me

Each user has one special saved query called Notify Me. It decides which newly eligible bulletins may produce push notifications. Normal saved Views do not automatically notify. Existing matches after a query change may produce one combined notification.

Notifications are a separate subsystem and delivery record, not bulletin rows.

Offline

The PWA caches the last synchronized graph and board state. Mutations may be created offline and queued. Pending, failed, conflicted, and synchronized states must be visible. Mutations require client-generated IDs, idempotency, and version-aware conflict handling. Blocking, deletion, and revoked authorization win over stale offline mutations.

5. Visibility, Safety, and Moderation

Viewer-controlled visibility

Visibility remains viewer-controlled. The author chooses an audience within allowed rules; the viewer chooses which eligible content they wish to see. Global engagement never decides visibility. Authorization is enforced before search, saved filters, or Notify Me evaluation.

Blocking

Blocking is a separate one-way safety invariant, not a trust value. A block causes mutual invisibility for product purposes: no graph routing, bulletin exposure, directed requests, notification delivery, or contact exposure. The blocked user is not explicitly notified. Cached and offline data must be invalidated on synchronization.

Private reporting and operator moderation

A user may privately report a specific bulletin as suspected misuse. Reporting immediately hides that bulletin for the reporter. The reporter may separately hide the author, disconnect, or block.

A report is not a public strike, reputation score, or automatic platform-wide penalty. Reports do not change visibility for other users. The reported person is not told who reported them.

Report reasons are constrained: commercial spam, misleading content, repeated irrelevant posting, harassment, unsafe or illegal content, or other. Operator review may inspect the reported bulletin and relevant metadata.

The operator may remove content, restrict bulletin creation, suspend an account, or disable an account where necessary. No report aggregation, report-weighted filtering, or automated three-strike system is required for v1.

6. Privacy and Encryption

v1 security baseline

- TLS protects all data in transit.
- Supabase and the deployment platform provide managed encryption at rest.
- User-upload storage buckets remain private and authorization-protected.

- Secrets live in platform secret stores, never source control or logs.
- Sensitive content is not copied unnecessarily into logs, analytics, queue payloads, or exception traces.
- Authorization and data minimization are the primary protections.

Bulletin content

Bulletins may contain private community information, but they are server-readable application data. The server must evaluate visibility, filters, Notify Me queries, notifications, synchronization, and moderation reports.

No E2EE claim

The product must not claim end-to-end encryption. Bulletin content is protected through transport encryption, managed storage encryption, strict authorization, least-privileged access, auditability, and deletion.

Deferred encryption work

Application-level field encryption, envelope encryption, KMS integration, and formal key-rotation workflows are deferred until a concrete threat, compliance obligation, or sensitive-data feature justifies them.

Do not duplicate authentication email into product tables without need. Avoid collecting precise addresses, unnecessary contact information, or freeform moderation notes. Data minimization is preferred over custom cryptography.

A future fixed-recipient private messaging feature may evaluate E2EE separately. It must not be silently mixed into the bulletin model.

7. Product Data Concepts

Concept	Meaning
User	Immutable internal ID, handle, display name, avatar, and optional independently visible fields.
Invitation	Opaque revocable token with lifecycle and audit history.
Connection	Mutual accepted relationship.
Directional Trust	Private per-user metadata attached to a connection.
Block	One-way safety invariant.
Bulletin	Typed short-lived content with visibility and lifecycle.
Tag	Suggested but extensible label.
Location	Provider-backed stable key and display label; not a user attribute.
View	Saved validated bulletin query and sort.
Notify Me	Single saved notification query.
Dismissal	Viewer-local bulletin state.
Report	Private claim about a specific bulletin.
Notification	Grouped and deduplicated delivery record.
Domain Event	Committed business fact.
Outbox Event	Durable asynchronous-delivery record.
Offline Mutation	Idempotent client-originated command awaiting synchronization.

8. Engineering Constitution

Domain-first feature modular monolith

Organize by business capability, not global technical layer. Expected modules include identity, connections, graph, bulletins, views, notifications, sync, storage, moderation, and audit. Each module owns transport definitions, application services, domain policies, persistence adapters, events, and tests.

SOLID and SRP

- Every class and module has one primary reason to change.
- Split services that coordinate unrelated use cases.
- Routers contain transport behavior, not business rules.
- Repositories contain persistence behavior, not product policy.
- Prefer focused interfaces over generic repositories and manager classes.
- Depend on abstractions at domain and application boundaries.
- Prefer composition over inheritance.
- Do not create ceremonial interfaces or classes with no meaningful boundary.

Layers and dependency direction

Default flow

tRPC procedure -> application service -> domain policy/entity -> repository interface -> repository implementation -> PostgreSQL

- Domain imports no transport, database, DI, or platform packages.
- Application services own use cases and transaction boundaries.
- Routers own authentication-context parsing, input validation, and output mapping only.
- Repositories own explicit persistence and query behavior only.
- Routers never call repositories directly.
- Repository implementations never emit domain events or make product decisions.

Dependency injection

Use constructor injection and a mature DI container. Resolve dependencies only in composition roots and explicit request or worker scopes. Business code must never resolve from the container. Unit tests should normally instantiate classes directly.

Proven patterns and hardened libraries

Use maintained, well-understood libraries and platform primitives before custom infrastructure. Custom infrastructure requires an ADR explaining why hardened options are unsuitable.

Do not invent routing, DI, validation, authentication, migrations, query building, logging, offline storage, event delivery, test runners, or query languages without demonstrated need.

Preferred baseline: React/Vite PWA, tRPC, Zod, Kysely, PostgreSQL, Supabase Auth/Storage, a mature DI container such as Awilix if runtime-compatible, Dexie, Workbox or vite-plugin-pwa, Vitest, Playwright, Testcontainers, and OpenTelemetry where useful. Verify current versions and runtime compatibility during implementation.

Commands, reads, and events

Use CQRS-lite, not event sourcing. Commands pass through services, domain policies, repositories, and transactions. Complex reads use purpose-built projections. Important successful state changes create past-tense domain events. Persist business state and outbox records atomically. Consumers process at least once and must be idempotent.

Database and querying

PostgreSQL is the source of truth. Recursive CTEs and checked-in SQL functions are first-class tools for graph traversal. Use a mature typed query builder for ordinary persistence and explicit SQL for database-native work. Do not hide graph behavior behind generic CRUD repositories.

Saved Views and Notify Me use a restricted, validated form of an existing read-only filter grammar over a server-defined authorized resource. User filters may narrow authorized data but never broaden access. Do not invent a custom query language or accept arbitrary read-only SQL.

Security architecture

Authentication is not authorization. Verify Supabase identity at the API boundary. Enforce authorization in the application/database boundary with least-privileged credentials and tested visibility functions or policies.

Do not assume a shared privileged database credential magically carries the user's Supabase JWT or RLS context. Document the authorization model before production deployment. Hidden data must never be returned to the client.

Deployment posture

Keep the application portable across a serverless edge deployment and a conventional Node host. Preferred starting options are Cloudflare frontend/API/queues/cron with Supabase after compatibility spikes, or Cloudflare frontend plus a normal Node API and worker on Railway or Hetzner if Worker constraints add friction.

Deployment must not affect domain or application architecture. Choose the simplest proven initial runtime after verifying tRPC, DI, Kysely transactions, database connectivity, web push, queues, and Supabase JWT validation.

Testing and architecture fitness

Prefer scenario tests over mocks of internal calls. High-risk matrices include graph reachability, hidden nodes, directional trust, visibility, blocking, bulletin authorization, report privacy, Notify Me matching, notification grouping, outbox retries, GDPR deletion, and offline replay/conflicts.

- No transport imports in domain or application logic.
- No database imports in domain.
- No SQL outside persistence adapters and checked-in SQL.
- No container resolution outside composition roots and scopes.
- No router-to-repository calls.
- No imports of another feature's concrete repositories.

9. Initial Vertical Slice

11. User authenticates through a magic link.
12. User opens or creates an invitation.
13. A second user joins and accepts a connection.
14. Each may assign private directional trust.
15. The graph renders the authorized relationship.
16. One user creates a Request bulletin.
17. The other sees it through an authorized Board query.
18. Notify Me produces one push notification when applicable.
19. The viewer dismisses or privately reports the bulletin.
20. The author archives the bulletin.
21. At least one mutation is exercised offline and replays idempotently.

This slice must include authorization tests and an outbox-backed asynchronous consequence. Do not build every module horizontally first.

10. Implementation Mandate for Claude Code

22. Read this document and the repository constitution before changing code.
23. Treat the prototype as product evidence, not production architecture.
24. Preserve the user experience and invariants; choose reasonable technical defaults without reopening settled product decisions.
25. Start with the vertical slice.
26. Produce an implementation plan and initial ADRs before consequential infrastructure choices.
27. Prefer proven patterns and hardened libraries.

28. Keep modules small, cohesive, and SRP-compliant.
29. Explain and document intentional deviations.
30. Build tests around observable behavior and security invariants.
31. Keep the repository runnable after each meaningful step.

Escalation threshold

Do not ask the product owner to decide ordinary technical details. Escalate only when a choice materially changes the user experience, privacy promise, safety model, product scope, or irreversible cost structure.